Part IV

8 Measuring the Power Spectrum

8.1 Density Contrast (δ)

We have learned how to assign mass of "particles" onto a grid to give us the density in each region. The power spectrum is related to "density contrast" which we call δ. The quantity δ is defined as:

$$\delta(\boldsymbol{r}) \equiv \frac{\rho(\boldsymbol{r}) - \bar{\rho}}{\bar{\rho}} \equiv \frac{\rho(\boldsymbol{r})}{\bar{\rho}} - 1$$

The quantity $\rho(\boldsymbol{r})$ is the density in each cell while the quantity $\bar{\rho}$ is the mean density. Overdense regions have a positive delta while underdense regions have a negative delta.

8.2 Power spectrum $P(\boldsymbol{k})$

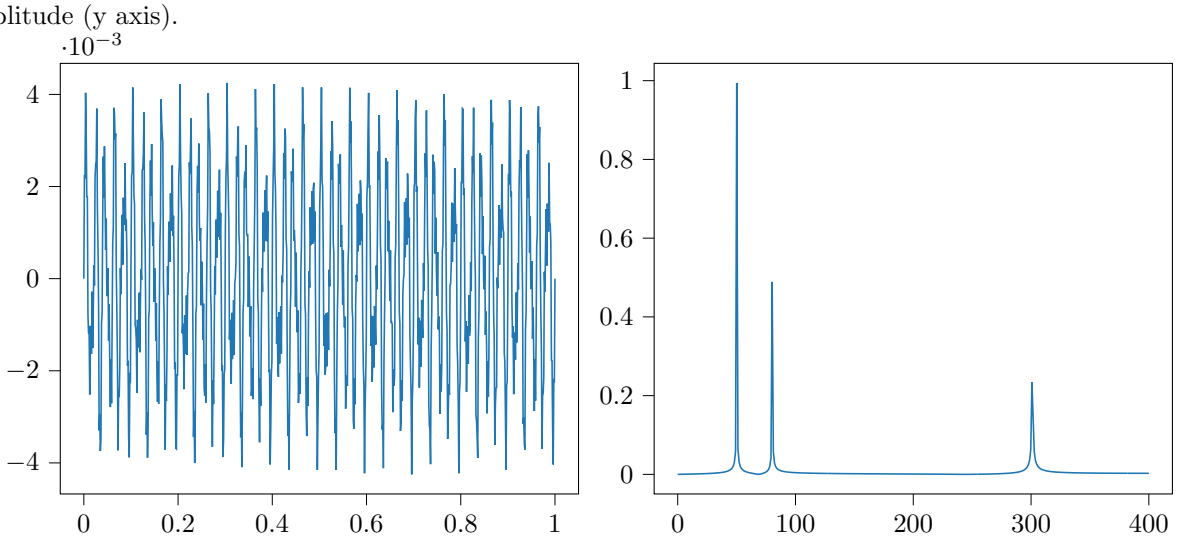
The goal is to measure the power spectrum which is related to the two-point correlation function. Fortunately now that we have δ the process is easy because:

$$P(k) \equiv \left\langle |\delta(\boldsymbol{k})|^2 \right\rangle$$

To convert from $\delta(\boldsymbol{r})$ to $\delta(\boldsymbol{k})$ we need only take the Fourier transform.

9 Fast Fourier Transform

The following plot shows the sum of three sine waves with different frequencies and amplitudes on the left side. The right side shows the Fourier transform which recovers the frequency (x axis) and amplitude (y axis).



10 FFTW

The FFTW library is open source, mature and widely used. It is usually present on HPC systems, and can usually be installed using the standard package manager,

- Ubuntu: `sudo apt install libfftw3-dev libfftw3-mpi-dev`
- OS/X: `brew install fftw`

For system where a package is not available, it can be downloaded from http://www.fftw.org. Documentation is available online as HTML or can be downloaded as a PDF.

10.1 Plans

Refer to section 2.1 of the FFTW User's Guide. There you will see the basic structure of a program that uses FFTW. It consists of the following steps.

1. Allocate memory for the input and output arrays.
2. Construct a plan (size and direction of the transform).
3. Execute the plan.
4. Destroy the plan.
5. Free the allocated memory.

10.1.1 Allocation and Alignment

There are a number of complications here. Refer to section 3.1 of the FFTW User's Guide. For optimal (and perhaps for correct) operation we need to allocate the memory in an aligned manner. This means that it should start on a boundary that is some multiple of the SIMD size (32 or 64 bytes depending on the machine). We are allocating large arrays, so there is no penalty for over-aligning — that is to say allocating memory on a 64 byte boundary when only 32 byte alignment is required. Up until now we have been letting Blitz++ manage the memory for us. Blitz++ does not allocate memory in a SIMD aligned fashion, but it is possible for us to allocate the memory ourselves, and give it to Blitz++.

Read section 2.3.7 'Creating an array from pre-existing data" in the Blitz++ User's Guide. Notice that there are constructors for the Blitz++ array that takes a pointer to memory as well as a policy. The policy can be one of the things.

1. `duplicateData` Allocates memory and makes a copy of the data you pass.
2. `deleteDataWhenDone` Uses the data, but deletes it (with `delete[]`) when the array is deleted.
3. `neverDeleteData` Uses the data provided, but does not delete it. You must delete it when done.

The FFTW User's Guide recommends that you call `fftw_malloc` but does not *require* it. One way to handle this is to allocate the data using `fftw_malloc` and create the Blitz++ array with `neverDeleteData`. This means that you **must** remember to delete it after you are finished, or there will be a memory leak in your program. Another options is to use the `deleteDataWhenDone` option when you create the array. When the array is destroyed, Blitz++ will automatically free the memory by calling `delete[]`. The problem here is that you can only `delete[]` memory that was allocated with `new[]` so for this to work we need to use `new[]` and not `fftw_malloc`.

It turns out that in C++17 you can specify the alignment when you invoke `new[]`. You can refer to operator new. This would look like the following.

```
#include <new>      // for std::align_val_t
...
float *data = new (std::align_val_t(64)) float[40]; // 512-bit alignment (AVX512)
```

This `data` pointer can now be used when you construct the Blitz++ array,

```
#include <new>                // for std::align_val_t
#include <cstdint>            // for std::uint64_t
#include "blitz/array.h"
...
int nGrid = 10;
auto N = std::uint64_t(1) * nGrid * nGrid * nGrid;
float *data = new (std::align_val_t(64)) float[N]; // 512-bit alignment (AVX512)
blitz::Array<float,3> array(data,
                            blitz::shape(nGrid,nGrid,nGrid),
                            blitz::deleteDataWhenDone);
```

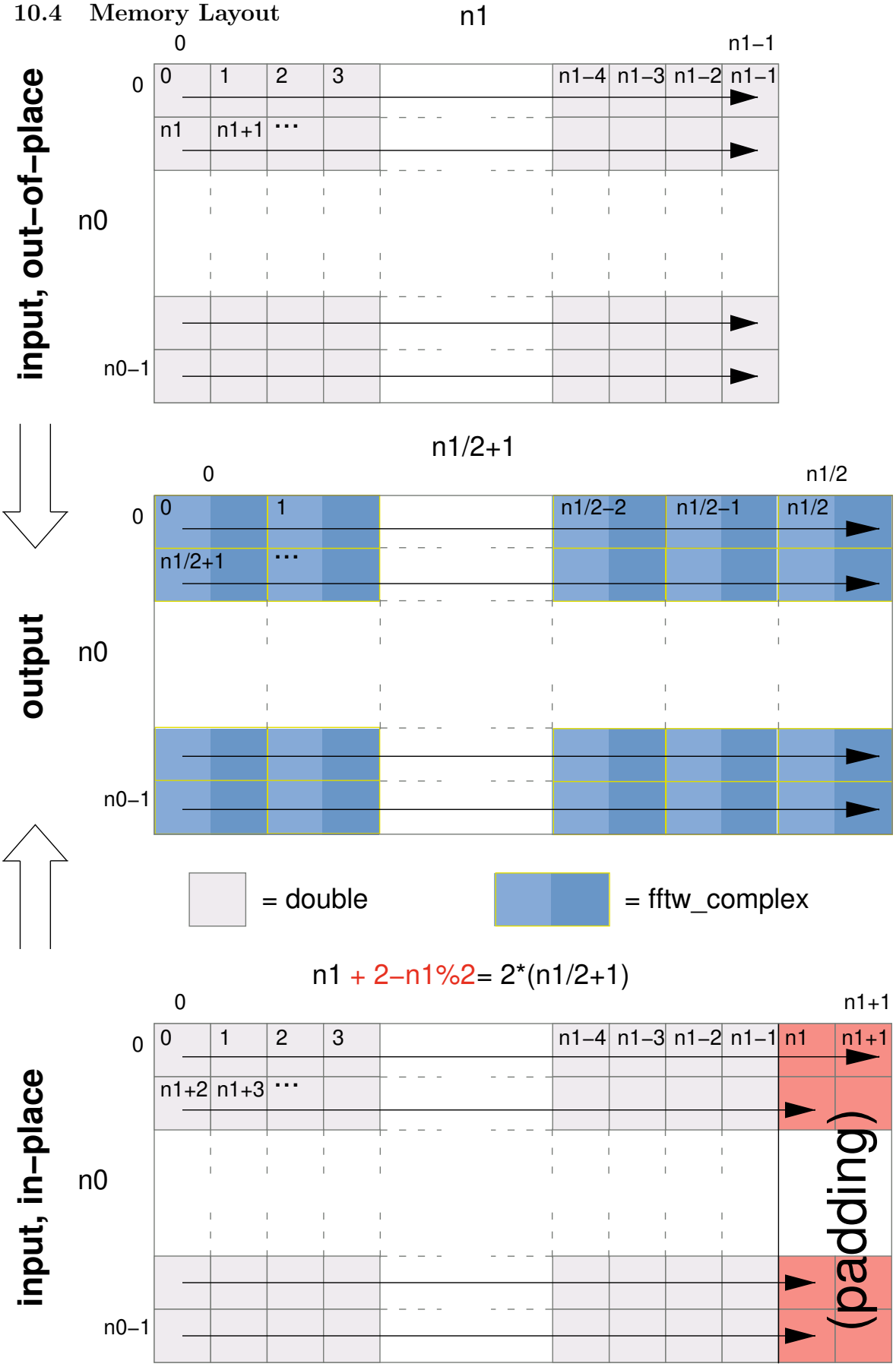
10.2 Wisdom

Page 18.

10.3 Algorithmic Complexity

The FFT is a divide and conquer algorithm and is formally $\mathcal{O}(n \log n)$ only for transforms that are a power of two. The FFTW library has been highly optimized and approaches this limit for many cases descibed in section 4.3.1 of the User's Guide. Generally it will handle transforms of size

$$2^a 3^b 5^c 7^d 11^e 13^f \tag{10}$$

where $e + f$ is 0 or 1 and the other exponents can be anything.

10.4 Memory Layout